ACTIVIDAD 4: MANIPULACIÓN DE CADENAS Y PUNTEROS EN C/C++ (Gestión de RAM)

Laboratorio de Programación (LPR) — 5° 3° A-B

Autores:

Sofia Mojica

Gabriel Zacarias

Mateo Roberts

Leonel De Armas

Docentes a cargo: York Mansilla / Yamil Ganduglia

1. Introducción

El presente informe documenta el desarrollo y los fundamentos teóricos de la **Actividad 4**, enfocada en la programación de bajo nivel mediante la manipulación de la memoria de acceso aleatorio (RAM). A diferencia de los lenguajes de alto nivel que abstraen la gestión de la memoria, C++ permite interactuar directamente con la arquitectura física del hardware. En este trabajo, se analiza la implementación de un sistema de control de accesos que interactúa con celdas de memoria referenciadas por direcciones hexadecimales únicas, utilizando punteros para optimizar el rendimiento del software.

2. Desarrollo Técnico (Glosario y Fundamentos de Arquitectura)

- **Memoria RAM:** Unidad de almacenamiento volátil y temporal donde el sistema operativo asigna bloques de celdas para alojar los datos binarios de los procesos en ejecución.
- **Dirección Hexadecimal:** Sistema de numeración posicional en base 16 (prefijo `0x`) que simplifica la lectura de los bits que componen el mapa de direcciones físicas del bus de datos.
- **Puntero (Pointer):** Variable escalar cuyo valor codificado corresponde de forma unívoca a la dirección física en la RAM de otra estructura de datos del sistema.
- **Operador de Dirección (`&`):** Operador unario que extrae la posición física en el hardware de memoria de cualquier variable asignada por el compilador.
- **Desreferenciar (`*`):** Instrucción de máquina que permite saltar de forma directa a la ubicación física apuntada por un puntero, otorgando privilegios de lectura o mutación sobre el valor original.

3. Distribución de Roles y Tareas del equipo

Integrante	Rol / Tarea Asignada
Sofía Mojica	Implementación de la lógica condicional de control de acceso por edad y manejo de lógica de punteros .
Gabriel Zacarías	Desarrollo del código de mapeo de direcciones hexadecimales (0x) y desreferenciación directa en la RAM
Mateo Roberts	Estructuración del repositorio base cadenaspunteros, control de versiones en Git y pruebas de terminal
Leonel De Armas	Diseño manual/digital del esquema físico de la RAM, resolución del cuestionario e informe en APA v7

4. Estructura del Repositorio del Proyecto

cadenaspunteros/
├── .gitignore <-- Archivo para que Git ignore carpetas de configuración (.vscode)
├── LICENSE <-- Archivo de licencia MIT del proyecto (Uso escolar)
├── README.md <-- Archivo explicativo del uso de punteros en la RAM
├── docs/ <-- Carpeta exclusiva para documentación
│ └── InformeProyectoCADENASPUNTEROS.pdf <-- ENTREGABLE (Informe PDF en Normas APA v7)
├── src/ <-- Carpeta para los códigos fuente del proyecto
│ └── main.cpp <-- Código fuente modular en C++
└── capturas/ <-- Carpeta para guardar las evidencias locales (Imágenes .png)
├── ejecucion_punteros.png <-- Captura de la terminal mostrando direcciones hexadecimales
└── esquema_ram.png <-- Diagrama de memoria hecho a mano en la carpeta

5. Cuestionario Técnico y Análisis de Memoria

¿Cuál es la ventaja de modificar una variable a través de un puntero en lugar de pasar una copia de la variable?

Al realizar una llamada de funciones común, el sistema operativo realiza una copia de la variable en el segmento de la pila (*Stack*), lo que duplica el consumo de espacio en la memoria RAM y añade sobrecarga a la CPU al copiar los datos. Al utilizar un puntero, el programa solo transfiere una dirección de 4 u 8 bytes (dependiendo de la arquitectura), modificando los datos directamente en su casillero original. Esto agiliza el rendimiento de los sistemas de software críticos y optimiza el uso del hardware.

¿Por qué las direcciones de memoria cambian cada vez que cerramos y volvemos a ejecutar el programa?

Esto ocurre debido a una medida de seguridad de los sistemas operativos modernos conocida como **ASLR (Aleatorización del Diseño del Espacio de Direcciones)**. Cada vez que se lanza el archivo executable `.exe`, el sistema operativo asigna un bloque dinámico de celdas en la RAM que se encuentren libres en ese instante de tiempo. Como la memoria se comparte con otros programas y procesos en segundo plano, la dirección física exacta (`0x...`) mutará inevitablemente en cada ciclo de ejecución.

¿Qué peligro técnico implica usar un puntero que apunta a una dirección inválida o no asignada?

Si un puntero no se inicializa (puntero salvaje) o apunta a un bloque de memoria que ya fue liberado, el programa intentará invadir celdas protegidas del sistema operativo o de otros procesos de software. Cuando la CPU detecta esta anomalía de hardware, aborta la ejecución del software de forma inmediata para proteger la integridad del sistema, provocando un error crítico conocido como **Segmentation Fault (Fallo de Segmentación)**
o cuelgue del programa.

6. Conclusión

El estudio práctico de la arquitectura de la memoria RAM a través del uso de punteros constituye un pilar indispensable para la formación de técnicos especialistas en desarrollo de software. Comprender los mecanismos de direccionamiento hexadecimal y el uso de los operadores de bajo nivel (`&` y `*`) provee las herramientas necesarias para escribir algoritmos eficientes en el uso de

memoria, sentando bases sólidas para el manejo avanzado de estructuras de datos dinámicas en los próximos años del ciclo técnico profesional.

7. Enlaces y Aportes Individuales de Integrantes

Cada integrante del grupo debe ingresar a su apartado correspondiente a

continuación y colocar su link individual (repositorio de GitHub).

1. ***Leonel De Armas***

https://github.com/tomasdearmas99-png/ACTIVIDAD-4-MANIPULACI-N-DE-CADENAS-Y-PUNTEROS-EN-C-C---LPR-5-3-A-B.git

2. *Sofia Mojica*

   https://github.com/Ailuna77/Actividad-4-Manipulacion-de-cadenas-y-punteros.git